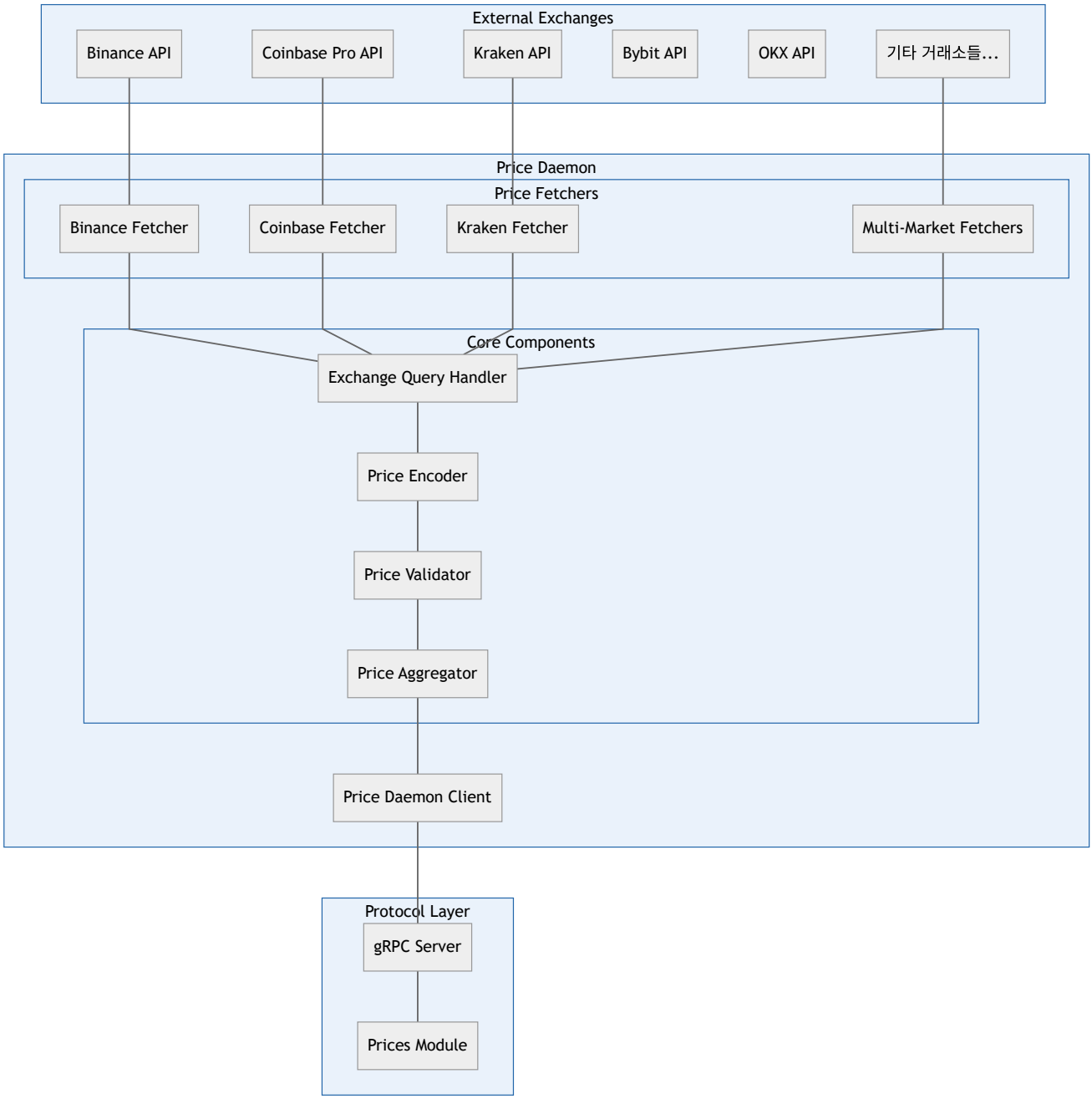
Price Daemon 구현 분석

개요

Price Daemon은 dYdX v4 Protocol의 핵심 구성 요소로, 외부 거래소에서 실시간 가격 데이터를 수집하여 블록체인의 가격 오라클에 전달하는 역할을 담당합니다. 이 데몬은 다중 거래소 API 연동, 실시간 가격 스트리밍, 데이터 검증 및 정규화, 그리고 gRPC를 통한 Protocol과의 통신을 처리합니다.

아키텍처 개요


External Exchanges
Binance API
Coinbase Pro API
Kraken API
Bybit API
OKX API
기타 거래소들...
Price Daemon
Price Fetchers
Binance Fetcher
Coinbase Fetcher
Kraken Fetcher
Multi-Market Fetchers
Core Components
Exchange Query Handler
Price Encoder
Price Validator
Price Aggregator
Price Daemon Client
Protocol Layer
gRPC Server
Prices Module

외부 거래소 API 연동 방식

지원하는 거래소

Price Daemon은 다음과 같은 주요 거래소들과 연동됩니다:

1. **Binance** - 다중 시장 지원 (Multi-Market)
2. **Coinbase Pro** - 단일 시장 쿼리
3. **Kraken** - 다중 시장 지원
4. **Bybit** - 다중 시장 지원
5. **OKX** - 다중 시장 지원
6. **Bitfinex** - 단일 시장 쿼리
7. **Bitstamp** - 단일 시장 쿼리
8. **Gate.io** - 다중 시장 지원
9. **Huobi** - 다중 시장 지원
10. **KuCoin** - 다중 시장 지원
11. **MEXC** - 다중 시장 지원
12. **Crypto.com** - 다중 시장 지원

API 연동 패턴

1. Multi-Market vs Single-Market 거래소

Multi-Market 거래소 (예: Binance)

```
// 한 번의 API 호출로 여러 시장의 가격을 동시에 조회
// URL 패턴: https://api.binance.com/api/v3/ticker/bookTicker?symbols=["BTCUSDT",
```

```
func (p *PriceFetcher) getNumQueriesPerTaskLoop() int {
if p.exchangeDetails.IsMultiMarket {
return 1 // 모든 시장을 한 번에 조회
}
// 단일 시장 거래소의 경우 시장 수만큼 쿼리 필요
return min(len(markets), queryLimit)
}
```

Single-Market 거래소 (예: Coinbase Pro)

```
// 각 시장마다 개별 API 호출 필요
// URL 패턴: https://api.exchange.coinbase.com/products/BTC-USD/ticker
func CoinbaseProPriceFunction(response *http.Response, tickerToExponent map[stri
// 단일 시장 응답 처리
ticker, _, err := price_function.GetOnlyTickerAndExponent(tickerToExponent, COIN
// ...
}
```

2. 거래소별 Price Function 구현

각 거래소는 고유한 API 응답 형식을 가지므로, 거래소별로 전용 Price Function이 구현됩니다:

Binance Price Function

```
type BinanceTicker struct {
```

```
Pair string `json:"symbol"`

AskPrice string `json:"askPrice"`

BidPrice string `json:"bidPrice"`

LastPrice string `json:"lastPrice"`

}


func BinancePriceFunction(response *http.Response, tickerToExponent map[string]i

var binanceTickers []BinanceTicker

json.NewDecoder(response.Body).Decode(&binanceTickers)

return price_function.GetMedianPricesFromTickers(binanceTickers, tickerToExponen

}
```

Coinbase Pro Price Function

```
type CoinbaseProTicker struct {

Pair string // 수동 설정

AskPrice string `json:"ask"`

BidPrice string `json:"bid"`

LastPrice string `json:"price"`

}


func CoinbaseProPriceFunction(response *http.Response, tickerToExponent map[stri

var coinbaseProTicker CoinbaseProTicker

json.NewDecoder(response.Body).Decode(&coinbaseProTicker)
```

```
coinbaseProTicker.Pair = ticker // 수동으로 페어 설정
return price_function.GetMedianPricesFromTickers([]CoinbaseProTicker{coinbasePro
}
```

API 요청 처리 메커니즘

1. URL 생성 및 요청 처리

```
func CreateRequestUrl(baseUrl string, tickers []string) string {
// "$" 플레이스홀더를 실제 티커 목록으로 치환
return strings.Replace(baseUrl, "$", strings.Join(tickers, ","), -1)
}

func (eqh *ExchangeQueryHandlerImpl) Query(ctx context.Context, exchangeQueryDet
// 1. 시장 ID를 거래소별 티커로 변환
tickers := make([]string, 0, len(marketIds))
for _, marketId := range marketIds {
config := exchangeConfig.MarketToMarketConfig[marketId]
tickers = append(tickers, config.Ticker)
}
// 2. API 요청 URL 생성
url := CreateRequestUrl(exchangeQueryDetails.Url, tickers)
// 3. HTTP GET 요청 실행
```

```
response, err := requestHandler.Get(ctx, url)

// 4. 응답 상태 코드 검증

if response.StatusCode == 429 {

return nil, nil, constants.RateLimitingError

}

// 5. 거래소별 Price Function으로 응답 파싱

prices, unavailableTickers, err := exchangeQueryDetails.PriceFunction(response,

}
```

2. 레이트 리미팅 및 에러 처리

```
// 429 Too Many Requests 처리

if response.StatusCode == 429 {

return nil, nil, constants.RateLimitingError

}


// 4xx, 5xx 에러 처리

if response.StatusCode < 200 || response.StatusCode > 299 {

return nil, nil, fmt.Errorf("%s %v", constants.UnexpectedResponseStatusMessage,

}
```

WebSocket 기반 실시간 가격 스트리밍 구현

스트리밍 아키텍처

Price Daemon은 HTTP REST API를 주기적으로 폴링하는 방식을 사용하며, WebSocket은 직접 구현되지 않습니다. 대신 다음과 같은 실시간성을 보장하는 메커니즘을 사용합니다:

1. 주기적 폴링 시스템

```
type Client struct {
tickers []*time.Ticker // 주기적 실행을 위한 티커들
stops []chan bool // 정지 신호를 위한 채널들
}

func (c *Client) newTickerWithStop(intervalMs int) (*time.Ticker, <-chan bool) {
ticker := time.NewTicker(time.Duration(intervalMs) * time.Millisecond)
c.tickers = append(c.tickers, ticker)
stop := make(chan bool)
c.stops = append(c.stops, stop)
return ticker, stop
}
```

2. 비동기 가격 수집

```
type PriceFetcher struct {
bCh chan<- *PriceFetcherSubtaskResponse // 버퍼링된 채널로 응답 전송
}
```

```
type PriceFetcherSubtaskResponse struct {
Price *types.MarketPriceTimestamp
Err error
}

// 각 Price Fetcher는 독립적인 고루틴에서 실행
func (p *PriceFetcher) fetchPrices() {
for {
select {
case <-ticker.C:
// 주기적으로 가격 수집 실행
response := p.queryExchange()
p.bCh <- response
case <-stop:
return
}
}
}
```

3. 실시간성 보장 메커니즘

빠른 폴링 주기

- 일반적으로 1-5초 간격으로 가격 데이터 수집

- 거래소별 레이트 리미트에 맞춰 조정

병렬 처리

- 각 거래소별로 독립적인 Price Fetcher 실행
- 고루틴을 통한 비동기 처리로 지연 최소화

버퍼링된 채널

- 가격 데이터를 버퍼링하여 일시적인 지연 흡수
- 백프레셔 방지를 통한 안정적인 데이터 흐름

가격 데이터 검증 및 정규화 프로세스

1. 데이터 검증 단계

입력 데이터 검증

```
// 구조체 태그를 통한 자동 검증
type BinanceTicker struct {
Pair string `json:"symbol" validate:"required"`
AskPrice string `json:"askPrice" validate:"required,positive-float-string"`
BidPrice string `json:"bidPrice" validate:"required,positive-float-string"`
LastPrice string `json:"lastPrice" validate:"required,positive-float-string"`
}
```

가격 유효성 검증

```
func GetMedianPricesFromTickers(tickers []Ticker, tickerToExponent map[string]in
```

```
for _, ticker := range tickers {

// 1. Ask, Bid, Last 가격이 모두 양수인지 확인

askPrice, err := strconv.ParseFloat(ticker.GetAskPrice(), 64)

bidPrice, err := strconv.ParseFloat(ticker.GetBidPrice(), 64)

lastPrice, err := strconv.ParseFloat(ticker.GetLastPrice(), 64)

// 2. Bid <= Last <= Ask 관계 검증

if bidPrice > lastPrice || lastPrice > askPrice {

// 비정상적인 가격 관계 감지

continue

}

// 3. 스프레드가 합리적인 범위 내인지 확인

spread := (askPrice - bidPrice) / lastPrice

if spread > maxAllowedSpread {

continue

}

}

}
```

2. 가격 정규화 프로세스

지수 기반 정규화

```
func ConvertStringPriceToUint64Price(stringPrice string, exponent int32) (uint64

// 1. 문자열을 float64로 변환
```

```
floatPrice, err := strconv.ParseFloat(stringPrice, 64)

// 2. 지수를 적용하여 정수로 변환

// 예: BTC 가격 50000.123, exponent -5 → 5000012300 (uint64)

multiplier := math.Pow(10, float64(-exponent))

adjustedPrice := floatPrice * multiplier

// 3. 오버플로우 검증

if adjustedPrice > math.MaxUint64 {

return 0, fmt.Errorf("price overflow")

}

return uint64(adjustedPrice), nil

}
```

중앙값 계산을 통한 이상치 제거

```
func (resolver types.Resolver) ResolvePrice(askPrice, bidPrice, lastPrice uint64

prices := []uint64{askPrice, bidPrice, lastPrice}

sort.Slice(prices, func(i, j int) bool { return prices[i] < prices[j] })

// 중앙값 반환 (3개 값 중 가운데 값)

return prices[1]

}
```

3. 가격 타임스탬프 관리

가격 유효성 시간 제한

```
const MaxPriceAge = time.Duration(30_000_000_000) // 30초


type MarketPriceTimestamp struct {
MarketId types.MarketId
Price uint64
LastUpdatedAt time.Time
}


func (mpt *MarketPriceTimestamp) IsExpired(currentTime time.Time) bool {
return currentTime.Sub(mpt.LastUpdatedAt) > MaxPriceAge
}
```

가격 업데이트 추적

```
func (eqh *ExchangeQueryHandlerImpl) Query(...) {
now := eqh.Now() // 현재 시간 획득
for ticker, price := range prices {
marketPriceTimestamp := &types.MarketPriceTimestamp{
MarketId: marketId,
Price: price,
LastUpdatedAt: now, // 가격 수집 시점 기록
}
marketPriceTimestamps = append(marketPriceTimestamps, marketPriceTimestamp)
```

```
}

}
```

gRPC를 통한 Protocol과의 통신 메커니즘

1. gRPC 서비스 정의

Price Feed Service 인터페이스

```
// price_feed.proto

service PriceFeedService {

rpc UpdateMarketPrices(UpdateMarketPricesRequest) returns (UpdateMarketPricesRes

}


message UpdateMarketPricesRequest {

repeated MarketPriceUpdate market_price_updates = 1;

}


message MarketPriceUpdate {

uint32 market_id = 1;

uint64 price = 2;

google.protobuf.Timestamp last_updated_at = 3;

}
```

```
message UpdateMarketPricesResponse {
// 응답 본문 (현재는 비어있음)
}
```

생성된 Go 코드 구조

```
type UpdateMarketPricesRequest struct {
MarketPriceUpdates []*MarketPriceUpdate `protobuf:"bytes,1,rep,name=market_price
}

type MarketPriceUpdate struct {
MarketId uint32 `protobuf:"varint,1,opt,name=market_id,json=marketId,proto3"`
Price uint64 `protobuf:"varint,2,opt,name=price,proto3"`
LastUpdatedAt time.Time `protobuf:"bytes,3,opt,name=last_updated_at,json=lastUpd
}
```

2. 클라이언트-서버 통신 흐름

Price Daemon → Protocol 데이터 전송

```
func (c *Client) sendPricesToProtocol(prices []*types.MarketPriceTimestamp) erro
// 1. 내부 타입을 gRPC 메시지로 변환
var marketPriceUpdates []*api.MarketPriceUpdate
for _, price := range prices {
```

```
update := &api.MarketPriceUpdate{
MarketId: uint32(price.MarketId),
Price: price.Price,
LastUpdatedAt: price.LastUpdatedAt,
}
marketPriceUpdates = append(marketPriceUpdates, update)
}
// 2. gRPC 요청 생성
request := &api.UpdateMarketPricesRequest{
MarketPriceUpdates: marketPriceUpdates,
}
// 3. Protocol의 gRPC 서버로 전송
ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()
_, err := c.grpcClient.UpdateMarketPrices(ctx, request)
return err
}
```

Protocol에서의 가격 데이터 수신 처리

```
func (s *PriceFeedServer) UpdateMarketPrices(
ctx context.Context,
req *api.UpdateMarketPricesRequest,
```

```
) (*api.UpdateMarketPricesResponse, error) {

// 1. gRPC 메시지를 내부 타입으로 변환

var priceUpdates []types.MarketPrice

for _, update := range req.MarketPriceUpdates {

priceUpdate := types.MarketPrice{

Id: update.MarketId,

Price: update.Price,

Exponent: getMarketExponent(update.MarketId),

}

priceUpdates = append(priceUpdates, priceUpdate)

}

// 2. Prices 모듈을 통해 블록체인 상태 업데이트

err := s.pricesKeeper.UpdateMarketPrices(ctx, priceUpdates)

if err != nil {

return nil, err

}

return &api.UpdateMarketPricesResponse{}, nil

}
```

3. 연결 관리 및 에러 처리

gRPC 연결 설정

```
type Client struct {
```

```
grpcConn *grpc.ClientConn

grpcClient api.PriceFeedServiceClient

}



func (c *Client) establishGRPCConnection() error {

// 1. gRPC 연결 옵션 설정

opts := []grpc.DialOption{

grpc.WithInsecure(),

grpc.WithTimeout(10 * time.Second),

grpc.WithKeepaliveParams(keepalive.ClientParameters{

Time: 10 * time.Second,

Timeout: 3 * time.Second,

PermitWithoutStream: true,

}),

}

// 2. Protocol gRPC 서버에 연결

conn, err := grpc.Dial(c.protocolGRPCAddress, opts...)

if err != nil {

return err

}

c.grpcConn = conn

c.grpcClient = api.NewPriceFeedServiceClient(conn)

return nil
```

```
}
```

연결 복구 및 재시도 로직

```
func (c *Client) sendPricesWithRetry(prices []*types.MarketPriceTimestamp) error
maxRetries := 3
backoffDuration := time.Second
for attempt := 0; attempt < maxRetries; attempt++ {
err := c.sendPricesToProtocol(prices)
if err == nil {
return nil
}
// gRPC 연결 에러인 경우 재연결 시도
if isConnectionError(err) {
c.logger.Warn("gRPC connection error, attempting to reconnect", "error", err)
c.grpcConn.Close()
if reconnectErr := c.establishGRPCConnection(); reconnectErr != nil {
c.logger.Error("Failed to reconnect to Protocol", "error", reconnectErr)
}
}
// 지수 백오프로 재시도
time.Sleep(backoffDuration * time.Duration(1<<attempt))
}
return fmt.Errorf("failed to send prices after %d attempts", maxRetries)
```

```
}
```

4. 성능 최적화 및 모니터링

배치 처리를 통한 효율성 향상

```
func (c *Client) batchPriceUpdates() {
batchSize := 100
batchTimeout := 1 * time.Second
var batch []*types.MarketPriceTimestamp
timer := time.NewTimer(batchTimeout)
for {
select {
case price := <-c.priceUpdateChan:
batch = append(batch, price)
// 배치 크기에 도달하면 즉시 전송
if len(batch) >= batchSize {
c.sendPricesWithRetry(batch)
batch = batch[:0]
timer.Reset(batchTimeout)
}
case <-timer.C:
// 타임아웃 시 현재 배치 전송
if len(batch) > 0 {
```

```
c.sendPricesWithRetry(batch)
batch = batch[:0]
}
timer.Reset(batchTimeout)
}
}
}
```

메트릭 수집 및 모니터링

```
func (c *Client) recordMetrics(prices []*types.MarketPriceTimestamp, err error)
// 전송된 가격 업데이트 수 기록
telemetry.IncrCounterWithLabels(
[]string{metrics.PricefeedDaemon, metrics.PriceUpdatesSent, metrics.Count},
float32(len(prices)),
[]gometrics.Label{},
)
// 에러 발생 시 에러 카운터 증가
if err != nil {
telemetry.IncrCounterWithLabels(
[]string{metrics.PricefeedDaemon, metrics.GRPCErrors, metrics.Count},
1,
[]gometrics.Label{metrics.GetLabelForStringValue(metrics.ErrorType, err.Error())
```

```
)

}

// gRPC 요청 지연 시간 측정

defer metrics.ModuleMeasureSince(

metrics.PricefeedDaemon,

[]string{metrics.PricefeedDaemon, metrics.GRPCRequestLatency, metrics.Latency},

time.Now(),

)

}
```

결론

Price Daemon은 dYdX v4의 가격 오라클 시스템의 핵심 구성 요소로서, 다음과 같은 주요 특징을 가집니다:

1. **다중 거래소 지원**: 12개 이상의 주요 거래소와 연동하여 가격 데이터의 신뢰성과 가용성을 보장
2. **실시간 데이터 수집**: 주기적 폴링과 비동기 처리를 통한 실시간성 확보
3. **강력한 데이터 검증**: 다단계 검증 프로세스를 통한 데이터 품질 보장
4. **효율적인 통신**: gRPC를 통한 고성능 Protocol 통신
5. **확장 가능한 아키텍처**: 새로운 거래소 추가가 용이한 플러그인 방식 설계

이러한 설계를 통해 Price Daemon은 탈중앙화 거래소의 핵심 요구사항인 정확하고 신뢰할 수 있는 가격 데이터를 안정적으로 제공합니다.